BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Number of items or other measure of problem size.

Q2. Record of intermediate variables/states.

Q3. Property maintained through a loop.

Q4. 128→64→32→16→8→4→2→1; seven halvings.

You Do pages · Independent questions

Q1. O(1) constant [2], O(n) linear [2], O(log n) logarithmic [2].

Q2. 64→32→16→8→4→2→1 [4], six halvings [2].

Q3. Binary requires sorted data [2]; preprocessing sort costs time [2]; single lookup may favour linear scan [1], repeated searches justify sorting [1].

Exit Ticket · Model answer

False. It describes asymptotic growth.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

LESSON 074 · Lesson 074 · Big O: Constant, Linear and Logarithmic

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What is an algorithm's input size n?

Q2. What is a trace table?

Q3. What is an invariant?

Q4. A sorted search set of 128 candidates is halved repeatedly to one. How many halvings? Show the sequence.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

O(1) — Constant asymptotic work.

O(n) — Linear growth.

O(log n) — Logarithmic growth.

asymptotic — Behaviour as size becomes large.

New learning

Observe — Big O describes the upper bound on how work grows with input size, not the precise milliseconds of an execution.

Explain — A fixed direct index operation is O(1), scanning n values is O(n), and repeatedly halving n is O(log n).

Evaluate — For n=64, a linear scan may inspect 64 items, while halving 64→1 takes six halvings. Sorting requirements and constants still matter.

Worked example

Classify loop performing one action for each item in n-element array.

1. Explain — The number of operations doubles when n doubles.

We Do · Practise with guidance

Classify a loop n=32; n=n/2 until n<=1.

```
O(log n); values 32,16,8,4,2,1: five halvings.
```

Explain

Each iteration cuts remaining size in half.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Match direct array access, full array scan and repeated halving with Big O. [6]

Question 2

For input size64, compare scan steps64 and repeated halving to1; calculate halvings. [6]

Question 3

Explain why O(log n) binary search may be unsuitable for unsorted data even if asymptotically faster than linear search. [6]

Exit Ticket · Answer before checking

Big O directly tells you the elapsed seconds of a program.